

Guide des bonnes pratiques Dataiku

Guide général — tous projets

1. Organisation des projets

Architecture en couches

Structurer systématiquement les projets en 3 couches séparées, chacune dans un projet Dataiku distinct :

Couche	Rôle	Ce qu'elle contient
DATASET	Extraction & normalisation	Connexion aux sources, nettoyage, schéma unifié
DESIGN	Traitement métier	Toute la logique, enrichissement, calculs
PRODUC	Publication	Copies simples vers les consommateurs aval

Pourquoi : chaque couche peut être rebuildée indépendamment, les droits d'accès sont gérés par projet, et la responsabilité de chaque projet est claire.

Zones dans le flow

- Créer une **zone par domaine fonctionnel** (ex : par pays, par source, par étape) pour que le flow reste lisible au-delà de 10 datasets
- Nommer les zones avec un libellé métier explicite, pas technique
- Respecter le sens gauche → droite dans le flow : sources à gauche, outputs à droite

2. Nommage

Datasets

```
{PROJET}_{domaine}_{objet}_{version}  
ex : COR_fr_event_prep_v1
```

Recipes

```
compute_{domaine}_{objet}  
ex : compute_fr_event_prep
```

Règles générales

- Utiliser uniquement des **minuscules et underscores** — pas d'espaces, pas de tirets, pas de majuscules mélangées

- Toujours suffixer avec la **version** (`_v1`, `_v2`) dès le début — bumper uniquement quand le schéma change de façon cassante pour les consommateurs
- Être cohérent sur toute la chaîne : le dataset `fr_event_v1` est produit par la recipe `compute_fr_event` et consommé dans `compute_fr_event_prep`

3. Datasets

Format de stockage

- Utiliser **Parquet avec compression SNAPPY** pour tous les datasets managés — meilleur compromis lecture/écriture/stockage
- Activer la **synchronisation avec le metastore** (Hive/Glue selon l'infra) pour que les datasets soient requêtables sans configuration supplémentaire
- Utiliser `writeBuckets: 1` pour les datasets de taille modérée — un seul fichier facilite le débogage

Schéma

- Toujours écrire avec `write_with_schema()` en Python — ne jamais laisser Dataiku inférer le schéma à la volée
- Définir les types de colonnes explicitement : éviter `string` pour des colonnes numériques ou des dates
- Ne jamais faire `SELECT *` dans une recipe SQL de production — lister les colonnes pour contrôler le schéma en sortie

Métriques

- Activer les **probes** `basic` et `records` sur tous les datasets intermédiaires et finaux — elles coûtent peu et permettent de détecter un dataset vide ou anormalement petit immédiatement après un build
- Configurer des **checks sur le** `record count` pour les datasets critiques : une alerte si le volume s'écarte de plus de X% de la valeur historique

4. Recipes Python (sans Spark)

Structure

- Placer toute la **logique métier** dans `lib/python/`, pas dans la recipe elle-même — la recipe est un orchestrateur, la lib est l'implémentation
- Toujours utiliser le garde `if __name__ == "__main__":` pour séparer le code Dataiku du code testable hors plateforme
- Une recipe = une responsabilité : ne pas mélanger extraction, transformation et écriture dans la même recipe

Lecture / écriture

- Lire avec `dataiku.Dataset("nom").get_dataframe()` — ne jamais accéder au path physique (S3, HDFS) directement
- Écrire avec `write_with_schema()` — jamais `write_dataframe()`
- Ne pas faire plusieurs `get_dataframe()` sur le même dataset dans la même recipe — lire une fois, stocker en variable

Paramétrage

- Utiliser `dataiku.get_custom_variables()` pour tous les paramètres qui peuvent changer entre exécutions (seuils, filtres, listes de variables)
- Ne jamais hardcoder des valeurs métier dans le code de la recipe

Qualité

- Écrire des **tests unitaires** dans `lib/python/` avec le préfixe `test_` pour chaque module — testables en dehors de Dataiku
- Utiliser un logger (ex : `logging.getLogger`) plutôt que des `print()` — les logs sont tracés dans l'historique des runs Dataiku

5. Recipes SQL

Structure des requêtes

- Utiliser des **CTEs** (`WITH ... AS`) pour décomposer les requêtes complexes en étapes nommées et lisibles
- Pour la déduplication, toujours utiliser `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` plutôt que des `GROUP BY` qui perdent les colonnes non agrégées
- Ajouter des **colonnes tie-breaker** dans les `ORDER BY` des fonctions fenêtres pour garantir un résultat déterministe à chaque exécution

Bonnes habitudes

- Préférer `INNER JOIN` quand la relation est obligatoire — ne pas utiliser `LEFT JOIN` par défaut
- Pousser les filtres `WHERE` au plus tôt dans les sous-requêtes pour réduire le volume avant les jointures
- Éviter les jointures sur des expressions calculées (`UPPER()`, `CAST()`, etc.) — elles empêchent l'utilisation des index

6. Recipes PySpark

Paradigme

- Utiliser les **transformations natives Spark** (`withColumn`, `join`, `groupBy`, `Window`) — ne jamais convertir en

Pandas avec `.toPandas()` sauf pour un agrégat de petit volume en toute fin de traitement

- Éviter les **UDFs Python** quand une fonction Spark native existe — les UDFs cassent l'optimisation Catalyst et sont ~10x plus lents
- Ne jamais itérer sur les lignes d'un DataFrame Spark avec une boucle `for` — c'est le pattern Pandas, pas Spark

Performance

- Utiliser `broadcast()` pour les petits DataFrames de référence (< quelques Mo) lors des jointures avec un grand DataFrame
- Repartitionner sur la clé de jointure avant les opérations coûteuses : `df.repartition(n, "cle_jointure")`
- Ne pas appeler `.count()`, `.collect()`, ou `.show()` en dehors du débogage — chaque appel déclenche un job Spark complet
- Toujours appeler `df.unpersist()` après un `df.cache()` ou `df.persist()` — ne pas laisser des DataFrames en cache après usage

7. Bibliothèques partagées (`lib/python/`)

- Chaque **module métier** a son fichier dédié (`merge.py`, `prepare.py`, `stats.py`) — pas de fichier fourre-tout
- Chaque module a son **fichier de tests** avec le préfixe `test_` (`test_merge.py`)
- Un fichier `utils.py` pour les fonctions transverses (logger, helpers génériques)
- Un fichier `dataiku_extender.py` pour les décorateurs/extensions Dataiku appliqués globalement
- **Règle d'or** : si du code est copié entre deux recipes, il doit aller dans la lib

8. Références cross-flow

- Toujours référencer un dataset d'un autre projet avec la notation complète : `"PROJECT_KEY.DATASET_NAME"`
- Les dépendances inter-projets ne vont **que dans un sens** : DATASET → DESIGN → PRODUCT — jamais l'inverse
- Ne jamais créer de dépendance circulaire entre projets

9. Scenarios & orchestration

Structure d'un scenario

1. **Étape d'initialisation** : définir les variables de run (timestamp, chemins temporaires, flags)

2. **Étape de build** : construire les datasets du flow
3. **Étape de finalisation** : upload des fichiers de logs/contrats, mise à jour de `last_run`

Bonnes pratiques

- Un scenario par projet, nommé clairement (`BUILD_DATASET`, `BUILD_DESIGN`, `RUN_PRODUCT`)
- L'enchaînement inter-projets se fait via un step `run_scenario` dans le scenario parent — jamais via des dépendances de datasets cross-flow dans une recipe
- Utiliser une variable `last_run` + condition `processing_month > last_run` pour rendre les scenarios **idempotents** — ne rebuild que si nécessaire
- Tagger les scenarios avec l'environnement cible (`prod_activated`, `pprod_activated`, `rd_activated`) pour contrôler leur activation sans toucher au code
- Configurer `proceedOnFailure: false` sur les steps critiques — un step qui échoue doit arrêter le scenario, pas le laisser continuer dans un état incohérent

Run condition

- Utiliser `runConditionType: RUN_IF_STATUS_MATCH` avec `[ "SUCCESS", "WARNING" ]` pour les steps normaux
- Utiliser `runConditionType: RUN_ALWAYS` uniquement pour les steps de nettoyage/upload qui doivent s'exécuter même en cas d'échec

10. Variables de projet

Type	Usage	Exemple
<code>standard</code>	Variables métier stables	<code>prel</code> , <code>variable_list</code> , filtres marché
<code>standard</code> (runtime)	Variables calculées au	<code>runtime</code> , <code>processing_month</code> , <code>last_ru</code>
<code>local</code>	Variables	Paths, flags de debug

- Ne jamais stocker de **secrets ou credentials** dans les variables de projet — utiliser les connexions Dataiku
- Toutes les variables modifiées dynamiquement pendant un run doivent être **réinitialisées en fin de scenario** pour ne pas polluer le prochain run

11. Gestion des erreurs et observabilité

- Implémenter des **data contracts** sur les datasets critiques (via Great Expectations ou équivalent) pour valider les hypothèses sur les données en entrée et en sortie

- Utiliser le **profiling de données** (whylogs ou équivalent) sur les datasets finaux pour détecter le drift entre deux exécutions
- Logger systématiquement le **volume de lignes** en entrée et en sortie de chaque transformation importante
- En cas d'échec d'une recipe, le dataset de sortie doit rester dans son état précédent (valide) — Dataiku garantit cela avec son mécanisme de commit transactionnel, mais ne pas désactiver ce comportement

Résumé visuel

SOURCE DB

| SQL recipe (extraction + dédup)



01_DATASET — lib: prepare.py, utils.py, tests

| Python recipe (normalisation, schéma unifié)

▼ (cross-flow: "01_DATASET.nom_dataset")

02_DESIGN — lib: merge.py, logic.py, stats.py, tests

| Python recipe (logique métier, enrichissement)

▼ (cross-flow: "02_DESIGN.nom_dataset")

03_PRODUCT — recipe = copie simple, zéro logique



CONSOMMATEURS (BI, API, autre projet)

quel est le difference entre l'utilisation de pandas , polars ou spark , quand utiliser quoi pour quel raison ? comment choisir ?

Pandas vs Polars vs Spark — Guide de choix

Les 3 outils en une phrase

Outil	En une phrase
Panda	La bibliothèque standard Python pour manipuler des données tabulaires en
Polars	Un remplaçant moderne de Pandas, beaucoup plus rapide, même paradigme
Spark	Un moteur distribué pour traiter des données qui ne rentrent pas sur une seule

La question centrale : où sont tes données ?

Taille du dataset

< 1 Go

→ Pandas ou Polars (en mémoire)



Ce n'est pas une règle absolue — ça dépend de la RAM disponible. La règle pratique : **si le dataset charge sans erreur mémoire et que `get_dataframe()` ne rame pas, tu n'as pas besoin de Spark.**

Comparaison détaillée

Pandas

Forces

- Écosystème immense : compatible avec quasiment toutes les libs Python (scikit-learn, matplotlib, statsmodels)
- Syntaxe connue de tous les data scientists — documentation abondante
- Idéal pour l'exploration interactive en notebook

Faiblesses

- **Single-threaded** : n'utilise qu'un seul cœur CPU
- Tout chargé **en RAM** : un dataset de 10 Go sur une machine avec 16 Go = problème
- Lent sur les grandes volumétries même si la RAM suffit

Quand l'utiliser

- Datasets < 500 Mo en pratique confortable
- Exploration, prototypage, notebooks d'analyse
- Quand tu consommes le résultat avec scikit-learn ou un outil qui attend du Pandas

Polars

Forces

- **Multi-threaded** nativement : utilise tous les cœurs de la machine automatiquement
- Moteur de calcul vectorisé (Apache Arrow) : 5x à 20x plus rapide que Pandas sur les mêmes opérations
- Même paradigme que Pandas (DataFrame, colonnes, filtres) — courbe d'apprentissage faible
- **Lazy evaluation** : Polars peut optimiser la requête avant de l'exécuter (comme SQL)
- Gestion mémoire bien plus efficace que Pandas

Faiblesses

- Moins intégré dans l'écosystème Dataiku que Pandas

- Certaines libs Python n'acceptent pas encore de DataFrame Polars en entrée (nécessite `.to_pandas()`)
- Moins de ressources en ligne que Pandas

Quand l'utiliser

- Datasets de 500 Mo à plusieurs Go sur une seule machine
- Pipelines de transformation répétitifs qui doivent être rapides
- Quand Pandas est trop lent mais que le volume ne justifie pas Spark

Spark (PySpark)

Forces

- **Distribué** : divise le traitement sur plusieurs machines en parallèle
- Traite des volumes **illimités** en théorie (To, Po)
- Lazy evaluation optimisée par le Catalyst Optimizer
- Intégration native avec les formats parquet, le metastore, S3

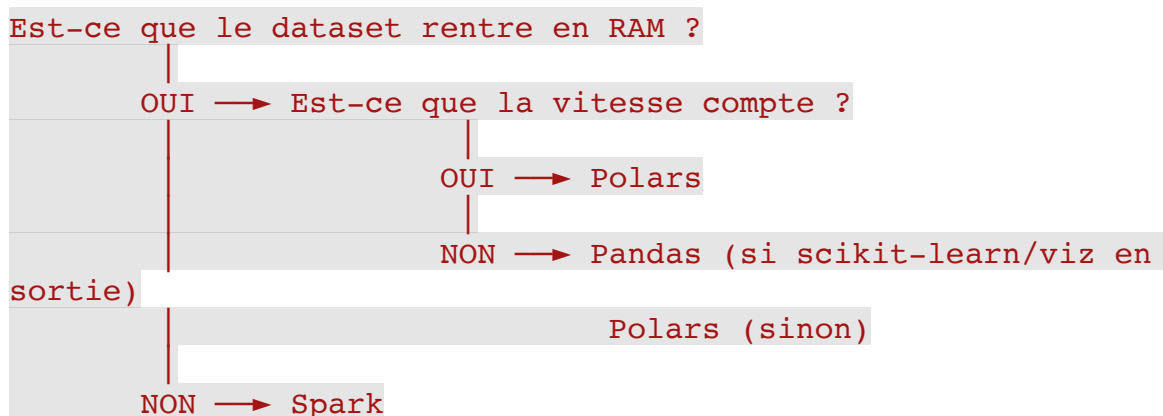
Faiblesses

- **Overhead de démarrage** : un job Spark prend 30s à 2 min juste pour démarrer le cluster
- Complexité : les erreurs sont difficiles à déboguer, le modèle mental est différent
- Certaines opérations "naturelles" en Pandas sont coûteuses en Spark (ex : itérer sur les lignes)
- Sur un petit dataset, Spark est **plus lent que Pandas** à cause de l'overhead réseau et de sérialisation

Quand l'utiliser

- Datasets qui ne rentrent pas en RAM sur une seule machine
- Traitements qui doivent tourner en parallèle sur plusieurs partitions
- Quand le cluster Spark est déjà là (Dataiku configuré avec Spark) et les données sont sur HDFS/S3 en volume

Règle de décision simple



Sur Dataiku spécifiquement

Situation	Choix recommandé
Recipe Python sur dataset < 1	Pandas — intégré
Recipe Python sur dataset 1–10	Polars — charger via PyArrow puis Polars, ou Pandas
Recipe Python sur dataset > 10	Spark via recipe PySpark
Transformation SQL simple	Recipe SQL — toujours plus rapide que Python pour
Entraînement ML	Pandas — scikit-learn, XGBoost, etc. attendent du
Agrégation sur table de	Recipe SQL ou Spark

Le piège le plus fréquent

"Mon code Pandas fonctionne en dev sur un échantillon, mais plante en prod sur les données complètes."

C'est le signe que la volumétrie a dépassé la RAM disponible. La solution n'est pas d'augmenter la RAM indéfiniment — c'est de choisir le bon outil dès le début en estimant la taille réelle des données de production.

Bonne habitude : avant d'écrire une recipe, vérifier le volume du dataset source dans les métriques Dataiku (nombre de lignes × taille estimée par ligne) et choisir l'outil en conséquence.

Résumé en une ligne par outil

- **Pandas** : simple, universel, limité à la RAM d'une machine, single-thread
- **Polars** : rapide, multi-thread, même paradigme que Pandas, limité à une machine
- **Spark** : distribué, illimité en volume, complexe, overhead important sur les petits datasets